



```

#cnc-regression-continuous
import pandas as pd
path = '/content/drive/MyDrive/Dataset/cncenergy.csv'
df = pd.read_csv(path)
from sklearn.linear_model import LinearRegression
o=LinearRegression()
o.fit(x,y)
o.predict([[1001]])
o.score(x,y)

#cnc-regression-continuous
import pandas as pd
path = '/content/drive/MyDrive/Dataset/cncenergy.csv'
df = pd.read_csv(path)
from sklearn.linear_model import Ridge
o=Ridge()
o.fit(x,y)
o.predict([[100]])
o.score(x,y)

#cnc-regression-continuous
import pandas as pd
path = '/content/drive/MyDrive/Dataset/cncenergy.csv'
df = pd.read_csv(path)
from sklearn.linear_model import Lasso
o=Lasso()
o.fit(x,y)
o.predict([[100]])
o.score(x,y)

#cnc-regression-continuous
import pandas as pd
path = '/content/drive/MyDrive/Dataset/cncenergy.csv'
df = pd.read_csv(path)
from sklearn.ensemble import RandomForestRegressor
o=RandomForestRegressor()
o.fit(x,y)
o.predict([[1001]])
o.score(x,y)

#cnc-regression-continuous
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
df = pd.read_csv('/content/drive/MyDrive/Dataset/cncenergy.csv')
df.columns = df.columns.str.strip()
df['label'] = (df['energyusage'] > df['energyusage'].median()).astype(int) #classifies energy usage as 0 = low or 1 = high
X = pd.get_dummies(df.drop(columns=['energyusage', 'label']))
y = df['label']
model = DecisionTreeClassifier().fit(X, y)
print("Prediction:", model.predict([X.iloc[296].values])) #for 296 it is classified

#cnc-regression-continuous
import pandas as pd
path = '/content/drive/MyDrive/Dataset/cncenergy.csv'
df = pd.read_csv(path)
from sklearn.ensemble import GradientBoostingRegressor
o=GradientBoostingRegressor()
o.fit(x,y)
o.predict([[1001]])
o.score(x,y)

#cnc-regression-continuous
import pandas as pd
path = '/content/drive/MyDrive/Dataset/cncenergy.csv'
df = pd.read_csv(path)
from sklearn.ensemble import AdaBoostRegressor
o=AdaBoostRegressor()
o.fit(x,y)
o.predict([[1001]])
o.score(x,y)

#cnc-regression-continuous
import pandas as pd
path = '/content/drive/MyDrive/Dataset/cncenergy.csv'
df = pd.read_csv(path)

```

```

from sklearn.svm import SVR
o=SVR()
o.fit(x,y)
o.predict([[1001]])
o.score(x,y)

#cnc-energy_usage
import pandas as pd
from sklearn.linear_model import LinearRegression
df = pd.read_csv('/content/drive/MyDrive/Dataset/cncenergy.csv')
X = df.drop(columns=['energyusage'])
y = df['energyusage']
model = LinearRegression().fit(X, y)
print("Predicted:", model.predict([X.iloc[0].values]))

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('/content/drive/MyDrive/Dataset/cncenergy.csv')
df.columns = df.columns.str.strip() # Clean column names

# Line Chart: Energy usage over index
plt.figure(figsize=(8, 4))
plt.plot(df['energyusage'], label='Energy Usage', color='green')
plt.title('Energy Usage Over Time')
plt.xlabel('Entry Index')
plt.ylabel('Energy Usage')
plt.grid(True)
plt.legend()
plt.show()

# Bar Chart: Average energy usage by material type
material_avg = df.groupby('Material type')['energyusage'].mean()
material_avg.plot(kind='bar', color='skyblue', figsize=(6,4))
plt.title('Average Energy Usage by Material Type')
plt.ylabel('Average Energy Usage')
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.show()

# Pie Chart: Machine type distribution
machine_counts = df['machine type'].value_counts()
machine_counts.plot(kind='pie', autopct='%1.1f%%', figsize=(5,5), startangle=140)
plt.title('Machine Type Distribution')
plt.ylabel('')
plt.show()

#pneumatic-classification-discrete
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df = pd.read_csv('/content/drive/MyDrive/Dataset/pneumatic.csv').dropna()
X = pd.get_dummies(df.drop('Valve_status', axis=1))
y = df['Valve_status']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = KNeighborsClassifier().fit(X_train, y_train)
print("Accuracy:", accuracy_score(y_test, model.predict(X_test)))

#pneumatic-classification-discrete
import pandas as pd
path = '/content/drive/MyDrive/Dataset/pneumatic.csv'
df = pd.read_csv(path)
from sklearn.linear_model import LogisticRegression
o=LogisticRegression()
o.fit(x,y)
o.predict([[100]])
o.score(x,y)

```

```

#pneumatic-classification-discrete
import pandas as pd
path = '/content/drive/MyDrive/Dataset/pneumatic.csv'
df = pd.read_csv(path)
from sklearn.ensemble import RandomForestRegressor
o=RandomForestRegressor()
o.fit(x,y)
o.predict([[100]])
o.score(x,y)

#pneumaticdecisiontree-FAULT FINDING
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
df = pd.read_csv('/content/drive/MyDrive/Dataset/pneumatic.csv').dropna()

# 3. Prepare features and target
X = pd.get_dummies(df.drop('Valve_status', axis=1)) # Features
y = df['Valve_status'] # Target: 'OPEN' or 'CLOSED'

# 4. Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Train classifier
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

# 6. Evaluate
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

#pneumaticgradientboost
import pandas as pd
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

df = pd.read_csv('/content/drive/MyDrive/Dataset/pneumatic.csv').dropna()
X = pd.get_dummies(df.drop('Valve_status', axis=1))
y = df['Valve_status']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = GradientBoostingRegressor().fit(X_train, y_train)
print("MSE:", mean_squared_error(y_test, model.predict(X_test)))

#pneumaticsvr
import pandas as pd
path = '/content/drive/MyDrive/Dataset/pneumatic.csv'
df = pd.read_csv(path)
from sklearn.svm import SVR
o=SVR()
o.fit(x,y)
o.predict([[100]])
o.score(x,y)

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('/content/drive/MyDrive/Dataset/pneumatic.csv')
df.columns = df.columns.str.strip() # Clean column names

# Line plot for the first numeric feature vs index
plt.plot(df.index, df[df.columns[0]])
plt.title("Line Graph of " + df.columns[0])
plt.xlabel("Index")
plt.ylabel(df.columns[0])
plt.grid(True)
plt.show()

# Bar graph of 'valve status' counts
valve_counts = df['Valve_status'].value_counts()

```

```
valve_counts.plot(kind='bar', color='skyblue')
plt.title("Bar Chart of Valve Status")
plt.xlabel("Valve Status")
plt.ylabel("Count")
plt.grid(axis='y')
plt.show()
```

```
# Pie chart of 'valve status' distribution
valve_counts.plot(kind='pie', autopct='%1.1f%%', startangle=90, figsize=(6,6))
plt.title("Pie Chart of Valve Status")
plt.ylabel("") # Remove y-axis label for cleaner look
plt.show()
```